

UNIVERSIDADE CATÓLICA DO SALVADOR
CURSO DE ENGENHARIA DE SOFTWARE

GLAUBER LOBO DOS SANTOS
EDUARDO DRAGONE PINHEIRO CORRÊA
MIGUEL ANTONIO TORRES DIOGO
VINICIUS BARBOSA VENEGEROLES DOS SANTOS
GABRIEL GARRIDO PEREIRA DA SILVA

BOOKVERSE:
DOCUMENTAÇÃO TÉCNICA

SALVADOR
2026

GLAUBER LOBO DOS SANTOS
EDUARDO DRAGONE PINHEIRO CORRÊA
MIGUEL ANTONIO TORRES DIOGO
VINICIUS BARBOSA VENEGEROLES DOS SANTOS
GABRIEL GARRIDO PEREIRA DA SILVA

BOOKVERSE:
DOCUMENTAÇÃO TÉCNICA

Documentação técnica apresentada ao curso de Engenharia de Software da Universidade Católica do Salvador como requisito da Atividade Avaliativa em Grupo da Segunda Unidade, sobre Padrões de Projeto, GRASP e Programação Orientada a Objetos.

SALVADOR
2026

Sumário

1 NOME E OBJETIVO DA APLICAÇÃO....4

2 PROBLEMA QUE A APLICAÇÃO RESOLVE....4

3 PRINCIPAIS FUNCIONALIDADES....4

4 PADRÕES DE PROJETO UTILIZADOS....5

5 ONDE E POR QUE CADA PADRÃO FOI IMPLEMENTADO....6

- **5.1 BUILDER (CRIACIONAL)....6**
- **5.2 FACTORY METHOD (CRIACIONAL)....6**
- **5.3 COMPOSITE (ESTRUTURAL)....6**
- **5.4 ADAPTER (ESTRUTURAL)....7**
- **5.5 STRATEGY (COMPORTAMENTAL)....7**
- **5.6 STATE (COMPORTAMENTAL)....7**
- **5.7 FACADE (COMPLEMENTAR)....8**
- **5.8 CHAIN OF RESPONSIBILITY (COMPLEMENTAR)....8**

6 PRINCÍPIOS GRASP APLICADOS....8

7 CONCEITOS DE PROGRAMAÇÃO ORIENTADA A OBJETOS UTILIZADOS....9

8 COMO EXECUTAR NO ECLIPSE....10

REFERÊNCIAS....10

1 NOME E OBJETIVO DA APLICAÇÃO

Bookverse é o sistema de vendas de uma livraria, desenvolvido em Java no Eclipse. O objetivo é controlar o ciclo completo de uma venda: montar o pedido (livros avulsos ou kits), pagar por Pix, cartão de crédito ou pagamento internacional em dólar, baixar o estoque e, quando necessário, cancelar o pedido com estorno do pagamento e devolução do estoque.

A aplicação roda em memória, sem banco de dados, e é executada pela classe principal BookverseMain, do pacote application, que demonstra os fluxos de compra, estorno e validação.

2 PROBLEMA QUE A APLICAÇÃO RESOLVE

Uma livraria que vende online precisa manter pedido, pagamento e estoque sempre coerentes entre si. Sem esse controle, aparecem situações como vender um livro sem estoque, cobrar um pedido que depois falhou, estornar duas vezes o mesmo pagamento ou cancelar um pedido sem devolver os livros ao estoque.

O problema fica mais difícil porque:

- cada forma de pagamento tem regras próprias (Pix e cartão em reais, pagamento internacional em dólar, feito por um provedor externo com outra interface);
- a livraria vende tanto livros avulsos quanto kits de livros, e um kit precisa baixar o estoque de cada livro que o compõe;
- o cancelamento depende de várias regras: o pedido e o pagamento precisam existir, pertencer um ao outro e estar em situações que permitam cancelar ou estornar.

O Bookverse centraliza essas regras e garante que pedido, pagamento e estoque mudem sempre juntos.

3 PRINCIPAIS FUNCIONALIDADES

1. **Cadastro de produtos e clientes:** criação com validação dos dados (código, nome, preço, CPF, e-mail) e armazenamento em repositórios em memória.
2. **Montagem de pedidos:** pedidos com livros avulsos ou kits; o preço de um kit é a soma dos livros que ele contém.
3. **Checkout:** valida o pedido e o estoque, cria o pagamento, processa pela forma escolhida, baixa o estoque e conclui o pedido. Se algo falhar depois do

pagamento aprovado, o estoque é devolvido e o pagamento é cancelado ou estornado.

4. **Três formas de pagamento:** Pix (BRL), cartão de crédito (BRL) e internacional (USD, processado por um provedor externo).
5. **Cancelamento e estorno:** valida as regras de cancelamento; pagamento pendente é cancelado, pagamento concluído é estornado, e o estoque volta quando o pedido já estava concluído.
6. **Controle de estoque:** impede vender acima do disponível e calcula o estoque de kits, inclusive kits dentro de kits.

A classe BookverseMain executa: compra com Pix, compra de kit com cartão, compra internacional, estorno da compra com cartão, três tentativas de cancelamento inválidas e cinco validações (produto sem preço, CPF inválido, pedido vazio, estoque insuficiente e transição de estado proibida).

4 PADRÕES DE PROJETO UTILIZADOS

O projeto implementa os seis padrões exigidos, dois de cada categoria, e dois padrões complementares que também resolvem problemas reais da aplicação, conforme o Quadro 1.

Quadro 1 – Padrões de projeto por categoria

Categoria	Padrão	Pacote e classes principais
Criacional	Builder	entities: Product.Builder, Order.Builder, OrderItem.Builder, Customer.Builder
Criacional	Factory Method	payment.factory: PaymentFactory, PixPaymentFactory, CardPaymentFactory, InternationalPaymentFactory
Estrutural	Composite	composite: ProductComponent, IndividualProduct, ProductBundle
Estrutural	Adapter	payment.adapter: PaymentGateway, DollarPaymentAdapter, DollarPaymentProvider
Comportamental	Strategy	payment.strategy: PaymentStrategy, RefundStrategy e as implementações Pix, Card e International
Comportamental	State	payment.state: PaymentState, PendingPaymentState, CompletedPaymentState, CancelledPaymentState, RefundedPaymentState
Complementar (estrutural)	Facade	facade: CheckoutFacade
Complementar (comportamental)	Chain of Responsibility	cancellation: OrderCancellationHandler e seus seis elos

Fonte: elaborado pelos autores (2026).

5 ONDE E POR QUE CADA PADRÃO FOI IMPLEMENTADO

5.1 BUILDER (CRIACIONAL)

- **Onde:** classes internas Builder de Product, Order, OrderItem e Customer. Os construtores dessas entidades são privados, então só é possível criá-las pelo método build().
- **Problema:** essas entidades têm vários campos do mesmo tipo (por exemplo, quatro String em Customer), o que torna construtores longos fáceis de usar na ordem errada. Além disso, nenhum objeto inválido pode existir no sistema.
- **Contribuição:** a criação fica legível e todas as validações ficam no build(), que lança InvalidDataException ou EmptyOrderException. Como os campos não têm setters, o objeto continua válido depois de criado.

5.2 FACTORY METHOD (CRIACIONAL)

- **Onde:** PaymentFactory é a classe criadora. O método createPayment(code, order) valida o pedido e calcula o valor, e chama o método fábrica instantiatePayment(...), implementado por PixPaymentFactory, CardPaymentFactory e InternationalPaymentFactory.
- **Problema:** a CheckoutFacade precisa criar o pagamento certo sem depender das classes concretas PixPayment, CardPayment e InternationalPayment. Cada tipo também tem uma moeda obrigatória.
- **Contribuição:** o checkout recebe uma PaymentFactory e não conhece os tipos concretos. Cada pagamento define sua própria moeda, então é impossível criar um Pix em dólar. Uma nova forma de pagamento entra com uma nova factory, sem alterar o checkout.

5.3 COMPOSITE (ESTRUTURAL)

- **Onde:** ProductComponent (componente), IndividualProduct (folha, um livro) e ProductBundle (composto, um kit que pode conter livros e outros kits).
- **Problema:** a livraria vende kits, e um item de pedido precisa aceitar tanto um livro quanto um kit. O preço e o estoque de um kit dependem dos livros que ele contém.
- **Contribuição:** OrderItem e InventoryService tratam livro e kit da mesma forma. Os métodos calculatePrice() e collectStock() são resolvidos por cada componente: a folha responde por si, o kit repassa para os filhos. Um novo tipo de componente não exige mudança no serviço de estoque.

5.4 ADAPTER (ESTRUTURAL)

- **Onde:** DollarPaymentAdapter implementa a interface interna PaymentGateway e usa o DollarPaymentProvider.
- **Problema:** o provedor de pagamentos em dólar tem outra interface: recebe o valor em centavos (long) e tem métodos com outros nomes (makeDollarPayment e refundDollarPayment).
- **Contribuição:** o adaptador converte o valor para centavos, valida a moeda e traduz as chamadas. As estratégias internacionais dependem só de PaymentGateway; trocar de provedor exige apenas um novo adaptador.

5.5 STRATEGY (COMPORTAMENTAL)

- **Onde:** interfaces PaymentStrategy e RefundStrategy, com uma implementação para Pix, cartão e internacional. O PaymentService recebe as estratégias em mapas e escolhe a certa pelo método de pagamento.
- **Problema:** processar e estornar são operações diferentes para cada forma de pagamento. Sem o padrão, o PaymentService teria um switch que cresceria a cada nova forma.
- **Contribuição:** cada algoritmo fica em sua própria classe e pode ser trocado ou adicionado sem alterar o PaymentService. As estratégias internacionais usam o Adapter, integrando os dois padrões.

5.6 STATE (COMPORTAMENTAL)

- **Onde:** Payment guarda um PaymentState e delega a ele os métodos complete(), cancelOrRefund(...), canBeCancelledOrRefunded() e isConsistentWith(...). Os estados são PendingPaymentState, CompletedPaymentState, CancelledPaymentState e RefundedPaymentState.
- **Problema:** o que se pode fazer com um pagamento depende da situação dele: um pendente é cancelado, um concluído é estornado, e um cancelado ou estornado não aceita mais nada.
- **Contribuição:** cada estado implementa as suas regras e devolve o próximo estado. Transições: PENDING → COMPLETED → REFUNDED e PENDING → CANCELLED; qualquer outra lança exceção. O PaymentService e as validações de cancelamento não fazem nenhuma comparação de status.

5.7 FACADE (COMPLEMENTAR)

- **Onde:** método checkout(order, paymentFactory, paymentCode) da CheckoutFacade, chamado pelo CheckoutController.
- **Problema:** concluir uma compra envolve OrderService, PaymentService, InventoryService e a factory de pagamento, em uma ordem específica e com compensação em caso de falha.
- **Contribuição:** o controller faz uma única chamada. Toda a sequência e a compensação (devolver estoque e cancelar ou estornar o pagamento) ficam em um só lugar.

5.8 CHAIN OF RESPONSIBILITY (COMPLEMENTAR)

- **Onde:** OrderCancellationHandler (classe base com setNext e handle) e seis elos, nesta ordem: OrderExistsHandler, PaymentExistsHandler, PaymentOwnershipHandler, OrderStatusHandler, PaymentStatusHandler e StatusConsistencyHandler. A corrente é montada pela CancellationChainFactory e usada pelo OrderCancellationService.
- **Problema:** cancelar um pedido exige várias regras independentes, e qualquer uma delas pode barrar o cancelamento.
- **Contribuição:** cada regra é uma classe com uma única responsabilidade. Uma regra nova é um novo elo adicionado na CancellationChainFactory, sem alterar os outros elos nem o serviço.

6 PRINCÍPIOS GRASP APLICADOS

O Quadro 2 relaciona cada princípio GRASP aos pontos do código em que ele aparece.

Quadro 2 – Princípios GRASP no Bookverse

Princípio	Onde aparece no Bookverse
Especialista na Informação	Product controla o próprio estoque (hasStock, decreaseStock, increaseStock); Order calcula o próprio total; OrderItem calcula o subtotal; cada PaymentState sabe o que pode ser feito naquele estado.
Criador	Order.Builder, pelo método addItem(produto, quantidade), cria o OrderItem que o pedido vai conter; as factories de pagamento criam os objetos Payment; a CancellationChainFactory cria e liga os elos da corrente.
Controlador	CheckoutController e CancellationController recebem as operações do sistema e as repassam para a fachada e o serviço, sem regra de negócio.

Princípio	Onde aparece no Bookverse
Baixo Acoplamento	As dependências são interfaces (PaymentStrategy, RefundStrategy, PaymentGateway, ProductComponent, PaymentState) e são injetadas pelo construtor na classe BookverseMain; o OrderCancellationService recebe a corrente pronta e não conhece os elos.
Alta Coesão	Cada classe tem uma responsabilidade: cada elo da corrente verifica uma regra, cada estratégia implementa um meio de pagamento, os repositórios só armazenam e os serviços só aplicam regras.
Polimorfismo	Estados, estratégias e componentes do Composite substituem condicionais por tipo: não há instanceof nem comparação de status por texto.
Invenção Pura	CheckoutFacade, CancellationChainFactory, os serviços e os repositórios não representam conceitos da livraria; existem para manter as entidades coesas e desacopladas.
Indireção	O Adapter fica entre as estratégias e o provedor em dólar; a Facade fica entre o controller e os serviços.
Variações Protegidas	A interface PaymentGateway protege o sistema de mudanças no provedor externo; PaymentFactory e os mapas de estratégias protegem o checkout da inclusão de novas formas de pagamento.

Fonte: elaborado pelos autores (2026).

7 CONCEITOS DE PROGRAMAÇÃO ORIENTADA A OBJETOS UTILIZADOS

O Quadro 3 apresenta os conceitos de Programação Orientada a Objetos e exemplos de onde aparecem no código.

Quadro 3 – Conceitos de POO no Bookverse

Conceito	Exemplos no código
Classes, objetos, atributos e métodos	Entidades Product, Order, OrderItem e Customer; os objetos são criados e usados na classe BookverseMain.
Encapsulamento e controle de acesso	Atributos private (e final quando não mudam); construtores privados nas entidades com Builder; sem setters em Product, OrderItem e Customer; o estoque só muda por decreaseStock e increaseStock; listas devolvidas com Collections.unmodifiableList.
Herança	PixPayment, CardPayment e InternationalPayment estendem Payment; as três factories estendem PaymentFactory; os seis elos estendem OrderCancellationHandler; as exceções estendem RuntimeException.
Composição	Order contém OrderItem; ProductBundle contém ProductComponent; Payment contém um PaymentState; CheckoutFacade é composta pelos serviços que usa.
Polimorfismo	PaymentService chama strategy.process(...) sem saber qual estratégia é; Payment chama state.complete() sem saber qual estado é; component.calculatePrice() funciona para livro e kit.
Abstração por interfaces	PaymentStrategy, RefundStrategy, PaymentGateway, PaymentState e ProductComponent.
Abstração por classes	Payment (método abstrato getPaymentMethod), PaymentFactory

Conceito	Exemplos no código
abstratas	(instantiatePayment) e OrderCancellationHandler (validate).
Sobrescrita	Anotação @Override em todos os estados, estratégias, factories, elos da corrente e nas subclasses de Payment.
Sobrecarga	ProductBundle.add(ProductComponent) e add(Product); Order.Builder.addItem(OrderItem) e addItem(ProductComponent, int).

Fonte: elaborado pelos autores (2026).

8 COMO EXECUTAR NO ECLIPSE

Requisitos: Eclipse IDE for Java Developers e JDK 21 (o projeto está configurado com JavaSE-21). Não há dependências externas.

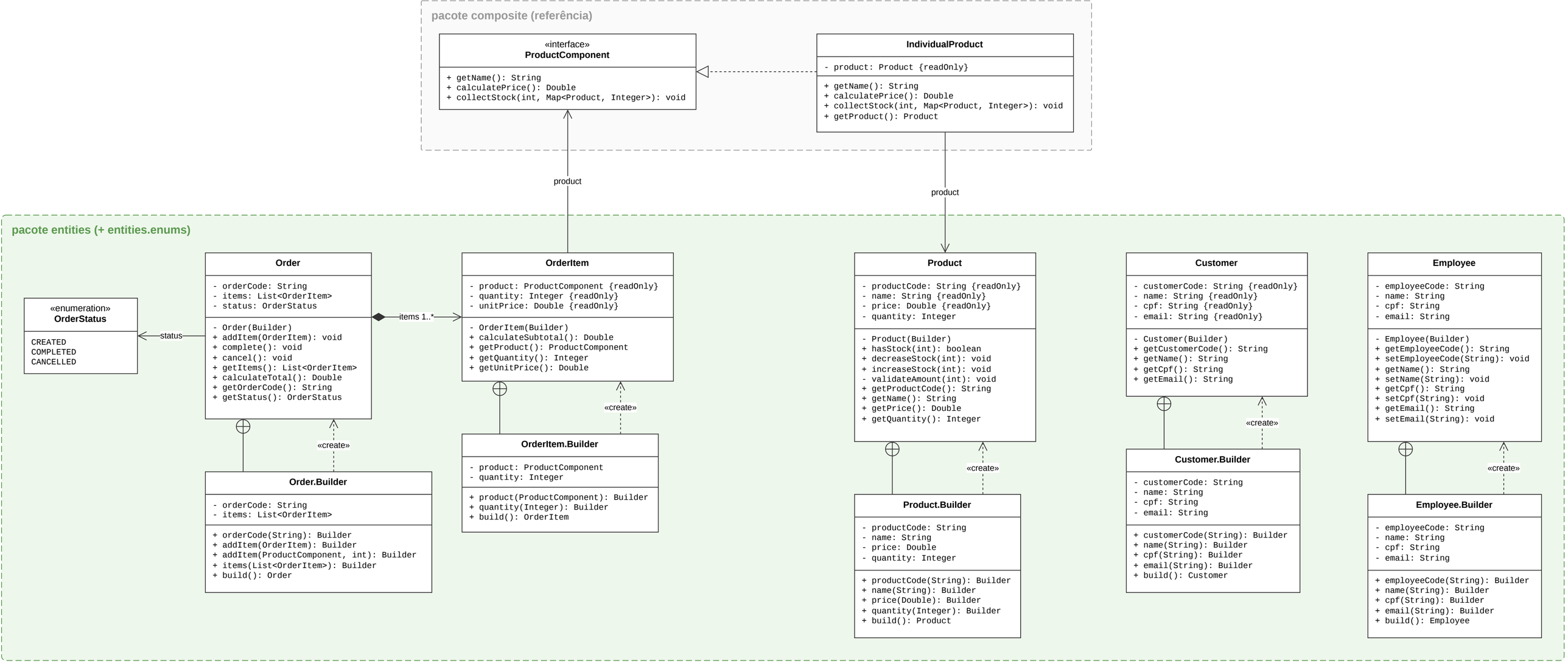
1. No Eclipse, abra **File → Import → General → Existing Projects into Workspace** e clique em **Next**.
2. Escolha **Select archive file** e selecione o arquivo .zip do projeto (ou **Select root directory** e a pasta Bookverse, se ele já estiver descompactado).
3. Marque o projeto **Bookverse** e clique em **Finish**.
4. No Package Explorer, abra o arquivo src/application/BookverseMain.java.
5. Clique com o botão direito no arquivo e escolha **Run As → Java Application**.
6. Acompanhe a saída no **Console**: as três compras, o estorno, os cancelamentos inválidos, as validações e o resumo final do estoque.

Se o Eclipse acusar erro de versão do Java, abra **Project → Properties → Java Build Path → Libraries** e selecione um JRE 21 instalado.

REFERÊNCIAS

GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. **Padrões de projeto**: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000.

LARMAN, Craig. **Utilizando UML e padrões**: uma introdução à análise e ao projeto orientados a objetos e ao desenvolvimento iterativo. 3. ed. Porto Alegre: Bookman, 2007.


pacote composite (referência)
«interface»
ProductComponent
+ getName(): String
+ calculatePrice(): Double
+ collectStock(int, Map<Product, Integer>): void
IndividualProduct
- product: Product {readOnly}
+ getName(): String
+ calculatePrice(): Double
+ collectStock(int, Map<Product, Integer>): void
+ getProduct(): Product
product
product
pacote entities (+ entities.enums)
«enumeration»
OrderStatus
CREATED
COMPLETED
CANCELLED
status
Order
- orderCode: String
- items: List<OrderItem>
- status: OrderStatus
- Order(Builder)
+ addItem(OrderItem): void
+ complete(): void
+ cancel(): void
+ getItems(): List<OrderItem>
+ calculateTotal(): Double
+ getOrderCode(): String
+ getStatus(): OrderStatus
items 1..*
OrderItem
- product: ProductComponent {readOnly}
- quantity: Integer {readOnly}
- unitPrice: Double {readOnly}
- OrderItem(Builder)
+ calculateSubtotal(): Double
+ getProduct(): ProductComponent
+ getQuantity(): Integer
+ getUnitPrice(): Double
«create»
«create»
Order.Builder
- orderCode: String
- items: List<OrderItem>
+ orderCode(String): Builder
+ addItem(OrderItem): Builder
+ addItem(ProductComponent, int): Builder
+ items(List<OrderItem>): Builder
+ build(): Order
OrderItem.Builder
- product: ProductComponent
- quantity: Integer
+ product(ProductComponent): Builder
+ quantity(Integer): Builder
+ build(): OrderItem
Product
- productCode: String {readOnly}
- name: String {readOnly}
- price: Double {readOnly}
- quantity: Integer
- Product(Builder)
+ hasStock(int): boolean
+ decreaseStock(int): void
+ increaseStock(int): void
- validateAmount(int): void
+ getProductCode(): String
+ getName(): String
+ getPrice(): Double
+ getQuantity(): Integer
«create»
Product.Builder
- productCode: String
- name: String
- price: Double
- quantity: Integer
+ productCode(String): Builder
+ name(String): Builder
+ price(Double): Builder
+ quantity(Integer): Builder
+ build(): Product
Customer
- customerCode: String {readOnly}
- name: String {readOnly}
- cpf: String {readOnly}
- email: String {readOnly}
- Customer(Builder)
+ getCustomerCode(): String
+ getName(): String
+ getCpf(): String
+ getEmail(): String
«create»
Customer.Builder
- customerCode: String
- name: String
- cpf: String
- email: String
+ customerCode(String): Builder
+ name(String): Builder
+ cpf(String): Builder
+ email(String): Builder
+ build(): Customer
Employee
- employeeCode: String
- name: String
- cpf: String
- email: String
- Employee(Builder)
+ getEmployeeCode(): String
+ setEmployeeCode(String): void
+ getName(): String
+ setName(String): void
+ getCpf(): String
+ setCpf(String): void
+ getEmail(): String
+ setEmail(String): void
«create»
Employee.Builder
- employeeCode: String
- name: String
- cpf: String
- email: String
+ employeeCode(String): Builder
+ name(String): Builder
+ cpf(String): Builder
+ email(String): Builder
+ build(): Employee

Facade (complementar)

CheckoutController

- checkoutFacade: CheckoutFacade

+ checkout(Order, PaymentFactory, String): Payment

CheckoutFacade

- orderService: OrderService
- paymentService: PaymentService
- inventoryService: InventoryService

+ checkout(Order, PaymentFactory, String): Payment

Serviços, repositórios e controllers

OrderService

- repository: OrderRepository

+ validateForCheckout(Order): void
+ save(Order): void
+ update(Order): void
+ findByCode(String): Order
+ delete(String): void
+ existsByCode(String): boolean

InventoryService

+ validateAvailability(List<OrderItem>): void
+ decreaseStock(List<OrderItem>): void
+ restoreStock(List<OrderItem>): void
- collectStock(List<OrderItem>): Map<Product, Integer>

PaymentService

- repository: PaymentRepository
- strategies: Map<String, PaymentStrategy>
- refundStrategies: Map<String, RefundStrategy>

+ validateNewPayment(Payment): void
+ processPayment(Payment): boolean
+ cancelOrRefund(Payment): boolean
- validatePayment(Payment): void

OrderCancellationService

- validationChain: OrderCancellationHandler
- orderService: OrderService
- paymentService: PaymentService
- inventoryService: InventoryService

+ cancelOrder(String, String): Payment

OrderRepository

- orders: Map<String, Order>

+ save(Order): void
+ findByCode(String): Optional<Order>
+ delete(String): void
+ existsByCode(String): boolean

CancellationController

- service: OrderCancellationService

+ cancelOrder(String, String): Payment

PaymentRepository

- payments: Map<String, Payment>

+ save(Payment): void
+ findByCode(String): Optional<Payment>
+ existsByCode(String): boolean

«use»

refundStrategies 0..*

strategies 0..*

validationChain

Strategy (comportamental)

«interface»
PaymentStrategy

+ process(Payment): boolean

«interface»
RefundStrategy

+ refund(Payment): boolean

PixPaymentStrategy

+ process(Payment): boolean

CardPaymentStrategy

+ process(Payment): boolean

InternationalPaymentStrategy

- gateway: PaymentGateway

+ process(Payment): boolean

PixRefundStrategy

+ refund(Payment): boolean

CardRefundStrategy

+ refund(Payment): boolean

InternationalRefundStrategy

- gateway: PaymentGateway

+ refund(Payment): boolean

gateway

gateway

Adapter (estrutural)

«interface»
PaymentGateway

+ processPayment(Payment): boolean
+ refundPayment(Payment): boolean

DollarPaymentAdapter

- provider: DollarPaymentProvider

+ processPayment(Payment): boolean
+ refundPayment(Payment): boolean
- validateCurrency(Payment): void

provider

DollarPaymentProvider

+ makeDollarPayment(long, String): boolean
+ refundDollarPayment(long, String): boolean

Chain of Responsibility (complementar)

nextHandler 0..1

OrderCancellationHandler

- nextHandler: OrderCancellationHandler

+ setNext(OrderCancellationHandler): OrderCancellationHandler
+ handle(CancellationContext): void {leaf}
validate(CancellationContext): void

«create»

«use»

CancellationContext

- orderCode: String {readOnly}
- paymentCode: String {readOnly}
- order: Order
- payment: Payment

+ getOrderCode(): String
+ getPaymentCode(): String
+ getOrder(): Order
+ getPayment(): Payment
~ setOrder(Order): void
~ setPayment(Payment): void

OrderExistsHandler

- repository: OrderRepository

validate(CancellationContext): void

PaymentExistsHandler

- repository: PaymentRepository

validate(CancellationContext): void

PaymentOwnershipHandler

validate(CancellationContext): void

OrderStatusHandler

validate(CancellationContext): void

PaymentStatusHandler

validate(CancellationContext): void

StatusConsistencyHandler

validate(CancellationContext): void

CancellationChainFactory

- CancellationChainFactory()
+ createDefaultChain(OrderRepository, PaymentRepository): OrderCancellationHandler

Factory Method (criacional)

PaymentFactory

+ createPayment(String, Order): Payment {leaf}
instantiatePayment(String, Order, Double): Payment

PixPaymentFactory

instantiatePayment(String, Order, Double): Payment

CardPaymentFactory

instantiatePayment(String, Order, Double): Payment

InternationalPaymentFactory

instantiatePayment(String, Order, Double): Payment

«create»

«create»

«create»

PixPayment

+ PixPayment(String, Order, Double)
+ getPaymentMethod(): String

CardPayment

+ CardPayment(String, Order, Double)
+ getPaymentMethod(): String

InternationalPayment

+ InternationalPayment(String, Order, Double)
+ getPaymentMethod(): String

Payment

- paymentCode: String {readOnly}
- order: Order {readOnly}
- amount: Double {readOnly}
- currency: String {readOnly}
- state: PaymentState

Payment(String, Order, Double, String)
+ getPaymentMethod(): String
+ markAsCompleted(): void
+ cancelOrRefund(RefundStrategy): void
+ canBeCancelledOrRefunded(): boolean
+ isConsistentWith(OrderStatus): boolean
+ getStatus(): PaymentStatus
+ getPaymentCode(): String
+ getOrder(): Order
+ getAmount(): Double
+ getCurrency(): String

state 1

State (comportamental)

«interface»
PaymentState

+ complete(): PaymentState
+ cancelOrRefund(Payment, RefundStrategy): PaymentState
+ canBeCancelledOrRefunded(): boolean
+ isConsistentWith(OrderStatus): boolean
+ getStatus(): PaymentStatus

«use»

«enumeration»
PaymentStatus

PENDING
COMPLETED
CANCELLED
REFUNDED

PendingPaymentState

+ complete(): PaymentState
+ cancelOrRefund(Payment, RefundStrategy): PaymentState
+ canBeCancelledOrRefunded(): boolean
+ isConsistentWith(OrderStatus): boolean
+ getStatus(): PaymentStatus

CompletedPaymentState

+ complete(): PaymentState
+ cancelOrRefund(Payment, RefundStrategy): PaymentState
+ canBeCancelledOrRefunded(): boolean
+ isConsistentWith(OrderStatus): boolean
+ getStatus(): PaymentStatus

CancelledPaymentState

+ complete(): PaymentState
+ cancelOrRefund(Payment, RefundStrategy): PaymentState
+ canBeCancelledOrRefunded(): boolean
+ isConsistentWith(OrderStatus): boolean
+ getStatus(): PaymentStatus

RefundedPaymentState

+ complete(): PaymentState
+ cancelOrRefund(Payment, RefundStrategy): PaymentState
+ canBeCancelledOrRefunded(): boolean
+ isConsistentWith(OrderStatus): boolean
+ getStatus(): PaymentStatus

order

Composite (estrutural)

«interface»
ProductComponent

+ getName(): String
+ calculatePrice(): Double
+ collectStock(int, Map<Product, Integer>): void

product

components 0..*

IndividualProduct

- product: Product {readOnly}

+ getName(): String
+ calculatePrice(): Double
+ collectStock(int, Map<Product, Integer>): void
+ getProduct(): Product

ProductBundle

- name: String {readOnly}
- components: List<ProductComponent>

+ add(ProductComponent): void
+ add(Product): void
+ remove(ProductComponent): void
+ getComponents(): List<ProductComponent>
+ getName(): String
+ calculatePrice(): Double
+ collectStock(int, Map<Product, Integer>): void

product

Entidades + Builder (criacional)

«enumeration»
OrderStatus

CREATED
COMPLETED
CANCELLED

status

Order

- orderCode: String
- items: List<OrderItem>
- status: OrderStatus

- Order(Builder)
+ addItem(OrderItem): void
+ complete(): void
+ cancel(): void
+ getItems(): List<OrderItem>
+ calculateTotal(): Double
+ getOrderCode(): String
+ getStatus(): OrderStatus

items 1..*

OrderItem

- product: ProductComponent {readOnly}
- quantity: Integer {readOnly}
- unitPrice: Double {readOnly}

- OrderItem(Builder)
+ calculateSubtotal(): Double
+ getProduct(): ProductComponent
+ getQuantity(): Integer
+ getUnitPrice(): Double

«create»

«create»

Order.Builder

- orderCode: String
- items: List<OrderItem>

+ orderCode(String): Builder
+ addItem(OrderItem): Builder
+ addItem(ProductComponent, int): Builder
+ items(List<OrderItem>): Builder
+ build(): Order

OrderItem.Builder

- product: ProductComponent
- quantity: Integer

+ product(ProductComponent): Builder
+ quantity(Integer): Builder
+ build(): OrderItem

Product

- productCode: String {readOnly}
- name: String {readOnly}
- price: Double {readOnly}
- quantity: Integer

- Product(Builder)
+ hasStock(int): boolean
+ decreaseStock(int): void
+ increaseStock(int): void
- validateAmount(int): void
+ getProductCode(): String
+ getName(): String
+ getPrice(): Double
+ getQuantity(): Integer

«create»

Product.Builder

- productCode: String
- name: String
- price: Double
- quantity: Integer

+ productCode(String): Builder
+ name(String): Builder
+ price(Double): Builder
+ quantity(Integer): Builder
+ build(): Product

Customer

- customerCode: String {readOnly}
- name: String {readOnly}
- cpf: String {readOnly}
- email: String {readOnly}

- Customer(Builder)
+ getCustomerCode(): String
+ getName(): String
+ getCpf(): String
+ getEmail(): String

«create»

Customer.Builder

- customerCode: String
- name: String
- cpf: String
- email: String

+ customerCode(String): Builder
+ name(String): Builder
+ cpf(String): Builder
+ email(String): Builder
+ build(): Customer

Employee

- employeeCode: String
- name: String
- cpf: String
- email: String

- Employee(Builder)
+ getEmployeeCode(): String
+ setEmployeeCode(String): void
+ getName(): String
+ setName(String): void
+ getCpf(): String
+ setCpf(String): void
+ getEmail(): String
+ setEmail(String): void

«create»

Employee.Builder

- employeeCode: String
- name: String
- cpf: String
- email: String

+ employeeCode(String): Builder
+ name(String): Builder
+ cpf(String): Builder
+ email(String): Builder
+ build(): Employee